

Technical Testing Report: Posts Explorer

Testing, Debugging, and Code Quality Assurance

Internship: YuvaIntern – ReactJS Development (Week 5)

Submitted By: Kaushal Kumar

Registration No: 24030485005

Institution: BIT Sindri, Jharkhand

Department: B.Tech Information Technology (2nd Year)

Submission Date: July 2, 2026

Project Overview

Posts Explorer is a ReactJS application originally developed during **Week 4** of the YuvaIntern ReactJS Internship. The application fetches and displays posts from the **DummyJSON API** using **Axios**, providing features such as real-time search, Read More/Show Less functionality, loading indicators, and error handling.

During **Week 5**, the focus shifted from adding new features to improving the application's reliability through automated testing, debugging, and code quality assurance. Using **Vitest** and **React Testing Library**, unit testing, integration testing, API mocking, and user interaction testing were implemented to verify that the application behaves correctly under loading, success, and error scenarios.

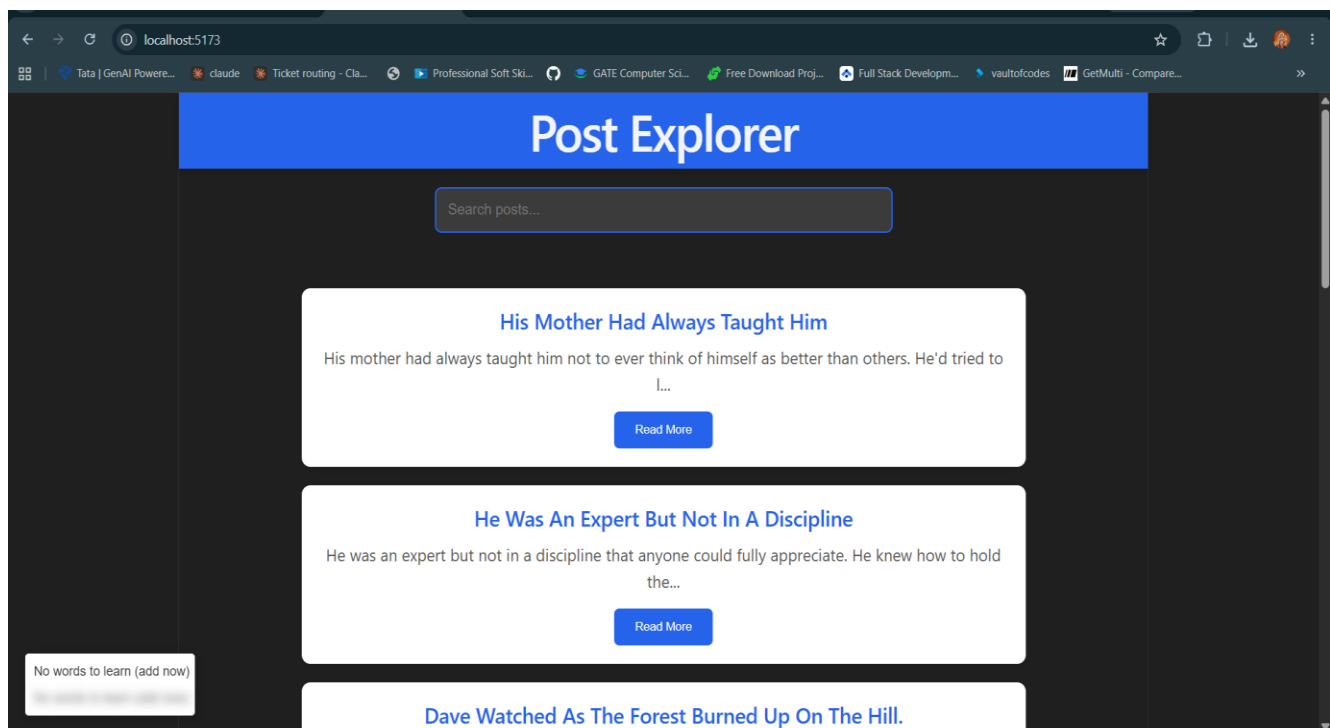


Figure 1: Posts Explorer Application User Interface

Objectives

- Integrate a modern testing framework (Vitest) into the existing Vite + React project.
- Write unit tests for each reusable component.
- Perform integration testing on the complete application flow.
- Mock API responses to test success, loading, and failure states without depending on network calls.
- Identify and fix bugs discovered during the testing process.
- Improve maintainability and confidence in the codebase before moving to further feature development.

Tech Stack

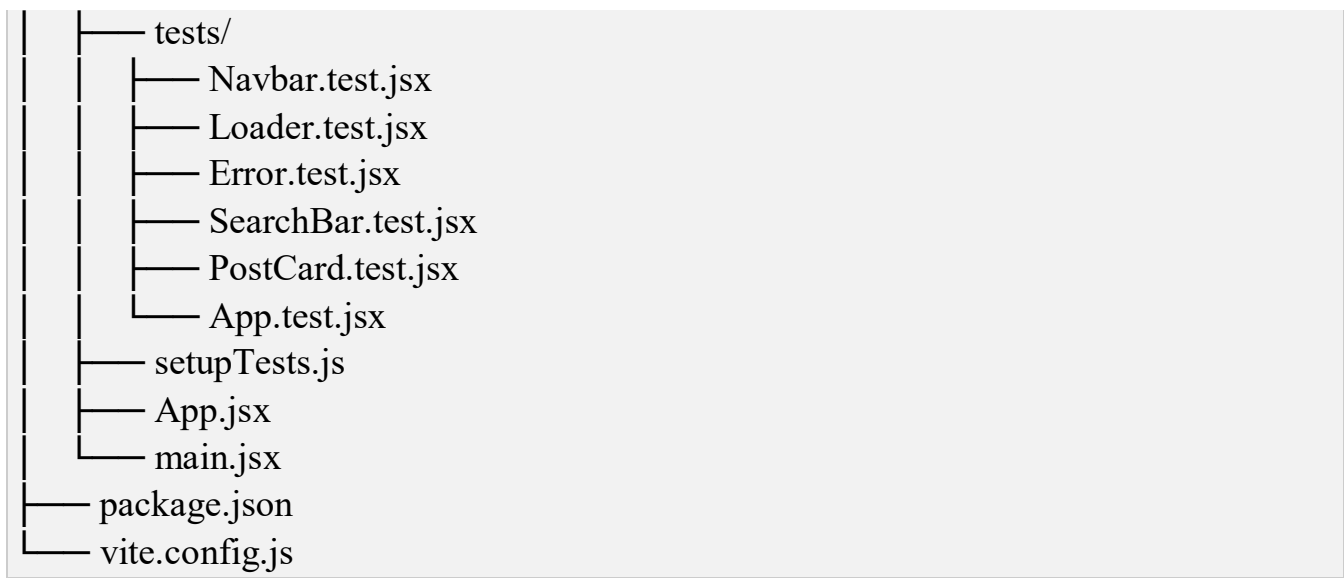
- ReactJS
- Vite
- JavaScript (ES6+)
- Axios
- CSS3
- DummyJSON API

Testing Tools

- Vitest — test runner, integrates natively with Vite
- React Testing Library — component rendering and interaction testing
- Jest DOM Matchers — extended assertions (toBeInTheDocument, etc.)
- User Event Library — simulating real user interactions (typing, clicking)

Project Structure

```
week5-post-explorer-testing/
├── src/
│   ├── components/
│   │   ├── Navbar.jsx
│   │   ├── SearchBar.jsx
│   │   ├── Loader.jsx
│   │   ├── Error.jsx
│   │   └── PostCard.jsx
│   ├── services/
│   │   └── api.js
```



Testing Strategy

1. Unit Testing

Each component was tested in isolation, independent of other components, to confirm it renders correctly and responds to props/events as expected. Components covered: Navbar, Loader, Error, SearchBar, PostCard.

2. Integration Testing

The complete application was tested by rendering the App component and verifying end-to-end behavior — how components work together when data is loading, successfully fetched, or fails to load.

Test Cases Implemented

Component	Test Cases	What Was Verified
Navbar	3	Renders successfully, heading displays correctly, navigation element exists
Loader	2	Loading message appears, heading renders correctly
Error	2	Error heading displays, dynamic error message renders correctly
SearchBar	2	Search input renders, user typing triggers the search function
PostCard	3	Post title renders, Read More button displays, button toggles to Show Less on click
App	3	Loader shown during fetch, posts rendered on successful response, Error component shown on failed request

Total Tests: 15

API Mocking

Instead of hitting the real DummyJSON API during test runs, the `fetchPosts()` service function was mocked using `vi.mock()`. This allowed simulation of three distinct scenarios:

- Successful response — posts array returned and rendered
- Loading state — request pending, Loader shown
- Failed request — error thrown, Error component shown

Mocking made the test suite fast, deterministic, and independent of internet connectivity — a test run doesn't fail just because the API is slow or temporarily down.

Debugging Process

#	Problem	Root Cause	Solution
1	No test files detected by Vitest	Files weren't following expected naming/location convention	Created properly named .test.jsx files inside a dedicated tests directory
2	Component import errors during test execution	Incorrect relative import paths in test files	Corrected paths to accurately point to component locations
3	Component rendering failed in tests	Required props were missing when rendering components in isolation	Passed mock props matching each component's expected interface
4	Unstable/flaky test runs	Tests were making real API calls, causing timing-dependent failures	Replaced real API calls with mocked responses via <code>vi.mock()</code>

Test Results

Metric	Value
Test Files	6
Total Tests	15
Passed	15
Failed	0
Success Rate	100%

All test suites passed on the final run, confirming that the core UI components and the primary application flow (loading → success/error) behave correctly under test conditions.

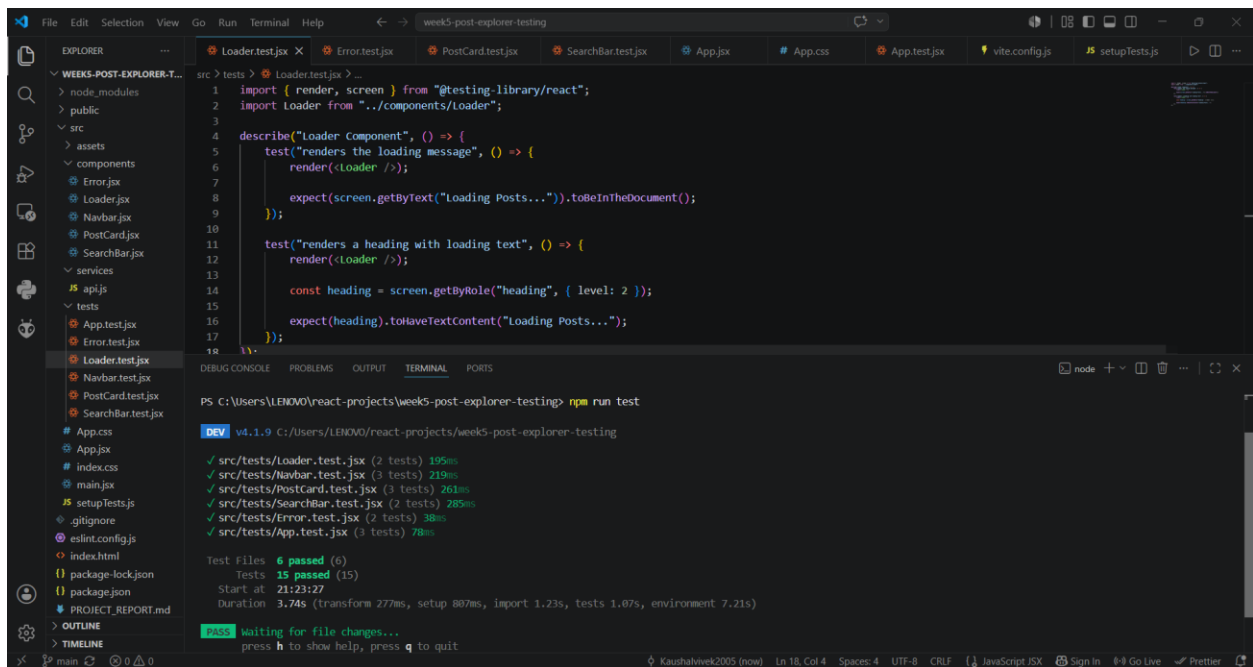


Figure 2: Successful execution of all 15 automated test cases using Vitest.

Learning Outcomes

- Writing unit tests for React components using React Testing Library
- Structuring integration tests to validate full application flow
- Mocking API calls with `vi.mock()` to isolate tests from network dependency
- Simulating user interactions (typing, clicking) with the User Event library
- Debugging test setup issues: file discovery, import paths, missing props
- Understanding the difference between unit-level and integration-level test coverage
- Practical experience with Vitest as a Vite-native test runner

Future Improvements

- Add test coverage reports to identify untested code paths
- Introduce snapshot testing for UI regression detection
- Add end-to-end testing with Cypress
- Add accessibility (a11y) testing
- Add performance testing
- Set up Continuous Integration (CI) with GitHub Actions to run tests automatically on every push

Conclusion

Week 5 shifted the project's focus from feature development to reliability and quality assurance. A total of 15 automated tests were written across 6 test files using Vitest and React Testing Library, covering both individual components and the application's end-to-end behavior under loading, success, and error conditions.

By mocking API calls, the test suite became fast and independent of network conditions. The debugging process — resolving file discovery, import path, missing props, and API-dependency issues — reinforced practical testing and debugging skills that will directly support more complex feature work in later weeks.

All planned unit tests, integration tests, and API mocking scenarios were completed successfully, achieving a 100% pass rate across all implemented test cases.

Author

Kaushal Kumar

B.Tech Information Technology

BIT Sindri, Jharkhand